SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA1208	**Course Title:**	Computer Architecture for Multicore Systems
CO Assessed:	CO5	**Assessment:**	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	**Total Marks:**	25
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	**GOPIKA P**	**Reg.No.:**	**192412306**
Department:	**ECE**	**Date:**	**2:09:2026**

ANNEXURE A

Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

*I **GOPIKA (192412306)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

GOPIKA P

ANSWERS – PSEUDOCODE WRITING TASK

QUESTION 1

DMA-Based Data Transfer from I/O Device to Memory

Problem Understanding

The algorithm must move a block of data from an I/O device to a memory buffer with minimum CPU involvement. The CPU should configure the DMA controller, allow the transfer to proceed independently, and respond only when the DMA controller generates a completion or error interrupt. This reduces CPU overhead and is suitable for continuous or high-volume I/O.

- Input: I/O device address, destination memory buffer, transfer size, transfer mode, and interrupt configuration.
- Output: data stored correctly in memory, along with completion or error status.
- Main control structures: initialization, validation, conditional checks, and an interrupt service routine.

Algorithm Design

The DMA controller is configured with the source device, destination memory address, transfer count, direction, and interrupt enable bit. After the transfer starts, the CPU is free to execute other tasks. When DMA completes or detects an error, it raises an interrupt. The ISR checks the status, clears the interrupt, updates the buffer state, and records the result.

Pseudocode

```
ALGORITHM DMA_IO_TO_MEMORY_TRANSFER
INPUT: device, memory_buffer, transfer_size
OUTPUT: transfer_status

INITIALIZE DMA_controller
VALIDATE device, memory_buffer, transfer_size
IF validation fails THEN
   RETURN ERROR_INVALID_PARAMETER
END IF

CONFIGURE DMA.source      <- device.data_register
CONFIGURE DMA.destination <- memory_buffer
CONFIGURE DMA.count       <- transfer_size
CONFIGURE DMA.direction   <- IO_TO_MEMORY
CONFIGURE DMA.mode        <- BLOCK_TRANSFER
ENABLE DMA completion and error interrupts
CLEAR previous DMA status flags
SET transfer_status <- BUSY
START DMA

WHILE transfer_status = BUSY DO
   CPU performs other useful tasks
END WHILE

INTERRUPT SERVICE ROUTINE DMA_ISR
   READ DMA.status
   IF DMA.error_flag = TRUE THEN
      CLEAR DMA error flag
      SET transfer_status <- ERROR
      SIGNAL transfer failure
```

```
    ELSE IF DMA.complete_flag = TRUE THEN
      CLEAR DMA complete flag
      VALIDATE received data/buffer status
      SET transfer_status <- COMPLETE
      SIGNAL transfer completion
    END IF
    RETURN FROM INTERRUPT
  END ISR

  RETURN transfer_status
  END ALGORITHM
```

Working Explanation

1. The CPU initializes the DMA controller only once for the current transfer.
2. The DMA controller becomes responsible for moving the data between the I/O device and memory.
3. The CPU does not poll every byte or word, so processor utilization is reduced.
4. A completion interrupt informs the CPU that the transfer has finished.
5. An error interrupt allows the system to recover without corrupting the buffer silently.

Why the Algorithm Minimizes CPU Involvement

The CPU performs configuration, not bulk copying. The DMA hardware handles the actual data movement, while a single interrupt can represent completion of a large transfer. The approach therefore scales better than byte-by-byte interrupt-driven I/O.

QUESTION 2

Prioritized Vectored Interrupt Handling for Multiple Devices

Problem Understanding

Multiple devices can request service at the same time. The interrupt system must identify the requesting device, prioritize vectored interrupt requests, and directly dispatch the corresponding service routine. Vectored interrupts reduce the need for software to search all devices after every interrupt.

- Input: pending interrupt requests, vector number, device priority, and interrupt vector table.
- Output: correct service routine executed for each accepted interrupt.
- High-priority requests are serviced before lower-priority requests when multiple requests are pending.

Algorithm Design

A programmable interrupt controller or equivalent hardware maintains pending requests and priorities. The CPU obtains the interrupt vector of the highest-priority pending device. The vector is used as an index into the service routine table. After service, the request is acknowledged and the next pending request is handled.

Pseudocode

```
ALGORITHM VECTORED_INTERRUPT_DISPATCH
INPUT: device_list, vector_table, priority_table
OUTPUT: interrupt_service_result

INITIALIZE interrupt_controller
FOR each device IN device_list DO
   REGISTER device.vector
   REGISTER device.priority
   ENABLE device interrupt
END FOR

LOOP FOREVER
   IF interrupt_controller.has_pending_request() = FALSE THEN
      CONTINUE
   END IF

   pending_set <- interrupt_controller.get_pending_requests()
   selected <- SELECT request IN pending_set WITH HIGHEST priority
   vector <- selected.vector

   SAVE minimal CPU context
   ACKNOWLEDGE selected request
   service_routine <- vector_table[vector]

   IF service_routine EXISTS THEN
      CALL service_routine(selected.device)
      SET interrupt_service_result <- SUCCESS
   ELSE
      LOG unknown vector condition
      SET interrupt_service_result <- ERROR_UNKNOWN_VECTOR
   END IF

   RESTORE CPU context
   RETURN FROM INTERRUPT
END LOOP
```

END ALGORITHM

Priority and Dispatch Rules

- Each interrupt source has a unique vector or vector index.
- Priority is evaluated before dispatch when several requests are pending.
- The service routine should perform only the urgent work required for the device.
- Long processing can be deferred to a background task so that higher-priority interrupts are not blocked.

Example

Assume three devices: a high-speed network interface with priority 1, a storage controller with priority 2, and a keyboard with priority 3. If all three interrupt requests arrive together, the vector for the network device is selected first, followed by storage and then keyboard. This provides deterministic priority-based handling.

QUESTION 3

Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O

Problem Understanding

The system must select an I/O addressing technique according to application requirements. Memory-mapped I/O places device registers in the processor address space and can use normal load/store instructions. I/O-mapped I/O uses a separate I/O address space and dedicated I/O instructions on architectures that support it. The selection algorithm compares required latency and bandwidth against configured thresholds.

- Input: required latency Lreq, required bandwidth Breq, available mapping modes, latency estimates, and bandwidth estimates.
- Output: selected mode - MEMORY_MAPPED_IO or IO_MAPPED_IO.
- Decision principle: choose the mode that satisfies both requirements; if both satisfy them, choose the one with lower weighted performance cost.

Decision Algorithm

A threshold-based method is simple and explainable. Memory-mapped I/O is preferred for very low latency or high-bandwidth register access when it provides the required performance. I/O-mapped I/O is selected when its overhead is acceptable and a separate I/O address space is preferred.

Pseudocode

```
ALGORITHM SELECT_IO_MAPPING
INPUT: Lreq, Breq
     MMIO_latency, MMIO_bandwidth
     PMIO_latency, PMIO_bandwidth
     latency_weight, bandwidth_weight
OUTPUT: selected_mode

FUNCTION satisfies(latency, bandwidth)
  RETURN (latency <= Lreq) AND (bandwidth >= Breq)
END FUNCTION

mmio_ok <- satisfies(MMIO_latency, MMIO_bandwidth)
pmio_ok <- satisfies(PMIO_latency, PMIO_bandwidth)

IF mmio_ok = TRUE AND pmio_ok = FALSE THEN
  selected_mode <- MEMORY_MAPPED_IO
ELSE IF pmio_ok = TRUE AND mmio_ok = FALSE THEN
  selected_mode <- IO_MAPPED_IO
ELSE IF mmio_ok = TRUE AND pmio_ok = TRUE THEN
  mmio_cost <- latency_weight * MMIO_latency
           + bandwidth_weight * (1 / MMIO_bandwidth)
  pmio_cost <- latency_weight * PMIO_latency
           + bandwidth_weight * (1 / PMIO_bandwidth)
  IF mmio_cost <= pmio_cost THEN
    selected_mode <- MEMORY_MAPPED_IO
  ELSE
    selected_mode <- IO_MAPPED_IO
  END IF
ELSE
  selected_mode <- NO_VALID_MODE
END IF
```

RETURN selected_mode
END ALGORITHM

Dynamic Operation

6. Read the current application requirements instead of using a fixed I/O mode for every device.
7. Estimate or obtain the performance characteristics of both addressing methods.
8. Reject any mode that fails either latency or bandwidth requirements.
9. If both are acceptable, compare a weighted cost and select the lower-cost option.
10. Re-evaluate the choice when the workload changes significantly.

Example

For a control device requiring very low response time, the algorithm is likely to select memory-mapped I/O when its measured latency is lower. For a slower peripheral where latency and bandwidth requirements are modest and I/O-space access is sufficient, I/O-mapped I/O may be selected. The algorithm remains dynamic because the choice is driven by requirements and measured characteristics.

QUESTION 4

Bus Arbitration for PCIe, USB, and Thunderbolt Devices

Problem Understanding

The system may contain multiple devices with different traffic characteristics. The arbitration algorithm must decide which request receives service next while respecting interface-specific priorities and preventing one requester from monopolizing service. The design uses a common logical scheduler with queues for PCIe, USB, and Thunderbolt traffic.

- PCIe traffic is organized through the host/root-complex and endpoint fabric; requests can be scheduled by priority and queue state.
- USB transfers are scheduled by the host controller, with timing requirements for periodic traffic.
- Thunderbolt devices can carry high-bandwidth traffic through a switched fabric and need fair bandwidth allocation.
- The pseudocode models a system-level arbitration policy above the interface-specific controller behavior.

Algorithm Design

The arbiter maintains separate queues, assigns weights, gives urgent or time-sensitive requests higher priority, and uses round-robin or weighted round-robin among comparable requests. A service quantum limits the number of bytes or transactions granted in one turn, improving fairness.

Pseudocode

```
ALGORITHM MULTI_INTERFACE_BUS_ARBITRATION
INPUT: request_queues[PCIe, USB, THUNDERBOLT]
      weight[PCIe, USB, THUNDERBOLT]
      service_quantum
OUTPUT: grants

INITIALIZE deficit[PCIe] <- 0
INITIALIZE deficit[USB] <- 0
INITIALIZE deficit[THUNDERBOLT] <- 0
last_selected <- THUNDERBOLT

LOOP FOREVER
   FOR each interface IN [PCIe, USB, THUNDERBOLT] DO
      deficit[interface] <- deficit[interface] + weight[interface]
   END FOR

   urgent <- FIND time-critical pending request
   IF urgent EXISTS THEN
      selected <- urgent.interface
   ELSE
      selected <- NEXT non-empty interface after last_selected
                  with maximum eligible deficit
   END IF

   IF selected = USB AND periodic transfer deadline is near THEN
      grant <- schedule required periodic USB transaction
   ELSE
      grant <- MIN(service_quantum, selected.pending_bytes)
   END IF

   SEND grant(selected, grant)
```

```
    UPDATE selected queue and counters
    deficit[selected] <- deficit[selected] - grant_cost
    last_selected <- selected

    IF selected queue becomes empty THEN
      RESET selected deficit to 0 when required
    END IF
  END LOOP
  END ALGORITHM
```

Fairness Mechanism

- Weighted service gives high-throughput interfaces enough service without permanently blocking USB or other traffic.
- Round-robin ordering among eligible queues prevents repeated selection of the same requester.
- A bounded service quantum prevents a single device from consuming the complete service opportunity.
- Urgent periodic transfers can be promoted when deadlines are close.

Result

The algorithm combines priority and fairness. It does not treat all interfaces as electrically identical; instead, it provides a scheduling policy that can sit above the interface controllers while allowing PCIe, USB, and Thunderbolt controllers to enforce their own protocol rules.

QUESTION 5

Hybrid I/O: Polling for Low-Speed Devices and DMA with Interrupts for High-Speed Devices

Problem Understanding

A hybrid design avoids wasting CPU time on high-speed data transfers while keeping the control logic for low-speed devices simple. Low-speed devices are checked periodically by polling because their event rate is small. High-speed devices use DMA for bulk movement and generate interrupts only for completion or error conditions.

- Low-speed devices: keyboard, simple sensors, status ports, or low-rate control devices can be polled periodically.
- High-speed devices: storage, network, data acquisition, or high-rate peripherals use DMA.
- CPU involvement is concentrated in polling intervals and DMA interrupt service routines rather than byte-level transfers.

Algorithm Design

The controller maintains a device configuration table. Each device is classified by speed. Polling tasks are scheduled using a timer or periodic loop. For high-speed devices, DMA descriptors are configured and started. A DMA completion interrupt signals the CPU to process or hand over the completed buffer.

Pseudocode

```
ALGORITHM HYBRID_IO_MANAGER
INPUT: device_table, memory_buffers
OUTPUT: device_status

INITIALIZE timer
INITIALIZE DMA_controller

FOR each device IN device_table DO
  IF device.speed = HIGH THEN
    CONFIGURE DMA source <- device.data_register
    CONFIGURE DMA destination <- memory_buffers[device]
    CONFIGURE DMA count <- device.transfer_size
    ENABLE DMA completion/error interrupt for device
    START DMA for device
  ELSE
    CONFIGURE poll_interval for device
    MARK device as POLL_MODE
  END IF
END FOR

LOOP FOREVER
  IF timer.poll_event() = TRUE THEN
    FOR each device IN device_table WHERE device.speed = LOW DO
      status <- READ device.status_register
      IF status indicates new data THEN
        READ available low-speed data
        STORE data in device buffer
      END IF
    END FOR
  END IF

  // High-speed completion handled asynchronously
```

```
    CONTINUE normal CPU work
END LOOP

INTERRUPT SERVICE ROUTINE HIGH_SPEED_DMA_ISR(device)
    READ DMA.status(device)
    IF DMA error THEN
        CLEAR error
        RECORD device_status <- ERROR
        RESTART or REPORT according to recovery policy
    ELSE IF DMA complete THEN
        CLEAR completion flag
        MARK buffer[device] <- READY
        NOTIFY data-processing task
        PREPARE next DMA descriptor
        START next transfer if data is continuous
    END IF
    RETURN FROM INTERRUPT
END ISR

    END ALGORITHM
```

Comparison of Polling and DMA with Interrupts

Aspect	Polling for Low-Speed	DMA + Interrupts for High-Speed
CPU work	Periodic status checking	Setup + completion/error ISR
Best use	Low event rate	Large or continuous data blocks
Data movement	CPU reads/writes data	DMA controller moves bulk data
Event handling	Timer-driven	Hardware interrupt-driven
Efficiency	Simple but wastes cycles if over-polled	Efficient for high data rates

Why the Hybrid Approach Works

Polling is acceptable when devices change slowly because the CPU can check them at a low frequency without significant overhead. High-speed transfers would create excessive polling or interrupt frequency, so DMA is used for bulk movement and interrupts are reserved for important state changes such as transfer completion or errors. This combination balances responsiveness, CPU efficiency, and implementation simplicity.

INTEGRATED ALGORITHM ANALYSIS

The five algorithms cover the major I/O communication mechanisms addressed in the assessment. DMA is used when large data movement should occur without continuous CPU participation. Vectored interrupts provide fast and direct service routine dispatch. Dynamic I/O mapping selects an addressing technique according to application requirements. Bus arbitration provides fair service among different interface classes. The hybrid model combines polling and DMA according to device speed.

Q	Main technique	Primary benefit	Key control structure
1	DMA transfer	Minimum CPU involvement	IF / WHILE / ISR
2	Vectored interrupts	Fast priority dispatch	FOR / IF / vector lookup
3	Dynamic I/O mapping	Requirement-based selection	IF / ELSE / function
4	Bus arbitration	Fair interface access	LOOP / FOR / priority / round-robin
5	Hybrid I/O	Balanced low/high-speed service	FOR / IF / timer / ISR

Completeness and Error Handling

- Invalid configuration values are rejected before a transfer or dispatch operation begins.
- DMA completion and error interrupts are handled separately.
- Unknown interrupt vectors are reported instead of silently ignored.
- The dynamic I/O selector can return NO_VALID_MODE when neither mapping satisfies the requested requirements.
- The bus arbiter uses bounded service quantities to reduce starvation.
- The hybrid manager can restart or report a DMA fault according to the configured recovery policy.

Conclusion

The proposed pseudocode designs convert the architectural requirements into clear, executable-style algorithms. Each solution uses appropriate control structures and explicitly handles normal operation as well as common error or edge cases. The overall approach reduces unnecessary CPU work, improves responsiveness, and provides a structured way to manage multiple I/O interfaces and interrupt sources.

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient